GUIA VISUAL 012

REDIRECIONAMENTO E PIPES

*Você guardou a varredura com `>` e os erros sumiram com o buffer da tela. Trocar para `2>&1 > arq` **não** conserta.*

012
GUIAS LINUX

TODO PROCESSO NASCE COM TRÊS CANAIS ABERTOS

`comando`

0 · stdin	←	teclado
1 · stdout	→	tela
2 · stderr	→	tela

Resultado e erro **misturados** na mesma tela — e é só a numeração que os separa.

`comando > saida.txt`

0 · stdin	←	teclado
1 · stdout	→	saida.txt
2 · stderr	→	tela

Só o descritor 1 mudou de destino. Redirecionar é reapontar um canal *antes* de o programa começar — e o programa nunca percebe.

A MESMA DUPLA, DUAS ORDENS, RESULTADOS OPOSTOS

`comando > log.txt 2>&1`

1. **`> log.txt`** — o descritor **1** passa a apontar para o arquivo.
2. **`2>&1`** — o **2** copia o destino *atual* do 1: o arquivo.

✓ resultado **e** erro no arquivo

`comando 2>&1 > log.txt`

1. **`2>&1`** — o **2** copia o destino atual do 1: a **tela**.
2. **`> log.txt`** — só o **1** muda para o arquivo.

✗ resultado no arquivo, erro **na tela**

`2>&1` **copia o destino *daquele instante* — não é vínculo.** O shell lê os redirecionamentos da esquerda para a direita, então a regra é: **o `2>&1` vem por último**, ou use `&>`. A ordem "errada" tem um uso legítimo e elegante: `comando 2>&1 > /dev/null | grep falha` manda **só os erros** para o cano, porque o `stderr` foi copiado para o cano antes de o `stdout` ser descartado.

AS FORMAS QUE SE USAM TODO DIA

FORMA	O QUE FAZ
`>` / `>>`	stdout sobrescrevendo / acrescentando
`2>` / `2>>`	Só o stderr
`&>` / `&>>`	Os dois no mesmo arquivo (Bash e Zsh; em `sh` use `> a 2>&1`)
`<`	Lê do arquivo. `wc -l < f` imprime só o número, sem o nome
`<< FIM`	*Here-doc*: texto até o marcador. Entre aspas (`<< 'FIM'`) não expande variável
`<<<`	*Here-string*: uma linha, sem `echo` e sem cano
`|` / `|&`	Cano só do stdout / com o stderr junto

`a | b | c`
Os três rodam **ao mesmo tempo**, não em sequência: o kernel põe um buffer entre cada par. É o que permite processar 100 GB com poucos MB de RAM.

`set -o pipefail` · `${PIPESTATUS[@]}`
Por padrão o status do pipeline é o do **último** comando: `falso | tee f` devolve **0**. Esse detalhe já mascarou backup quebrado por meses.

`echo linha | sudo tee -a /etc/hosts > /dev/null`
A resposta certa para `sudo ... > arquivo`, que falha porque **quem redireciona é o seu shell**, não o sudo. O `tee` grava e repassa.

`diff <(ls /etc) <(ls /etc/apt)`
Substituição de processo: entrega um caminho como `/dev/fd/63` onde o comando exige **nome de arquivo**. Bash e Zsh, não `sh`.

`grep --line-buffered` · `stdbuf -oL` · `unbuffer`
Filtro ao vivo que "trava" é *buffering*: a libc usa buffer por linha na tela e **por bloco** no cano. Não é bug — é otimização atrapalhando.